

Access and Alignment of Data in an Array Processor

DUNCAN H. LAWRIE, MEMBER, IEEE

Abstract—This paper discusses the design of a primary memory system for an array processor which allows parallel, conflict-free access to various slices of data (e.g., rows, columns, diagonals, etc.), and subsequent alignment of these data for processing. Memory access requirements for an array processor are discussed in general terms and a set of common requirements are defined. The ability to meet these requirements is shown to depend on the number of independent memory units and on the mapping of the data in these memories. Next, the need to align these data for processing is demonstrated and various alignment requirements are defined. Hardware which can perform this alignment function is discussed, e.g., permutation, indexing, switching or sorting networks, and a network (the *omega* network) based on Stone's shuffle-exchange operation [1] is presented. Construction of this network is described and many of its useful properties are proven. Finally, as an example of these ideas, an array processor is shown which allows conflict-free access and alignment of rows, columns, diagonals, backward diagonals, and square blocks in row or column major order, as well as certain other special operations.

Index Terms—Alignment network, array processor, array storage, conflict-free access, data alignment, indexing network, *omega* network, parallel processing, permutation network, shuffle-exchange network, storage mapping, switching network.

I. INTRODUCTION

IN THIS paper we will discuss the design of a memory system for an array processor. By an array processor we mean a collection of N arithmetic-logic units (ALU's) driven by a common instruction decoding unit (e.g., Illiac IV). We shall assume that the ALU's are connected to a collection of M memory units by some networks as illustrated in Fig. 1. In this system data come from the memory units in the form of vectors of N or fewer words, or N -vectors. The first network does the necessary rearranging or *alignment* of the elements of the N -vector so that each element can be paired in some ALU with the corresponding element of a second N -vector. This second N -vector may already be in the ALU's or it may be the next N -vector to be fetched from the memory. After some pairwise operation has been done on the two N -vectors, the resulting N -vector may be aligned for storage by the second network and stored in the memory units.

We can picture this system operating in a pipelined way: successive N -vectors being fetched, aligned, computed, aligned, and stored all at the same time. In order

to do this, each subsystem in Fig. 1 must normally function in a constant time and each subsystem must be balanced with all other subsystems. That is, in the time required to perform a vector operation, the memory must be capable of fetching two N -vector operands and storing an N -vector result, the first switch must be capable of aligning two separate N -vectors, and the second switch must perform a result alignment. Other paths than those shown in Fig. 1 might also be desirable, e.g., a path from the output of the ALU's to the input of the first switch in order to bypass the memories for intermediate results.

Of course other arrangements are possible, and the results we present in this paper are not necessarily restricted to the arrangement shown. We feel this arrangement is of interest due to the potential for high utilization of all the hardware.

One of the most significant factors in determining the performance of an array processor (aside from the parallelism of the program) is the system's capability for providing data vectors at a rate matched to the processor rate. It is not sufficient to provide a memory whose raw bandwidth is matched to the bandwidth of the processor. We must also insure that data access normally occurs without memory conflicts and that operand pairs can be brought together in the right ALU's for processing. The primary concern of this paper is to design a memory system which can do this.

We will begin by describing a memory organization which allows conflict-free access to various kinds of N -vectors. Then we discuss the problem of data alignment and present some data alignment networks.

II. MEMORY ORGANIZATION ACCESS AND ALIGNMENT REQUIREMENTS

Memory organizations which allow conflict-free access to various slices of data have been discussed previously in the literature. For example, memories which allow access to either words or bit slices have been described for the STARAN [2] and OMEN [3] computers.

In this paper we are interested in accessing certain slices of arrays, e.g., rows and columns. One technique is to *skew* the data [4] as illustrated in Fig. 2. Here the 4×4 array a is stored in four independent memory units. By skewing the data in this fashion we can access, without conflict, any row or column of the array. But access to the main diagonal would involve a memory conflict since a_{00} and a_{22} both lie in the same memory unit (cf. Kraska [5] and Millstein [6]).

Budnik and Kuck [7] described a different scheme

Manuscript received December 12, 1973; revised June 2, 1975. This work was supported by the NSF under Grant DCR73-07980 A02.

The author is with the Department of Computer Science, University of Illinois, Urbana, Ill. 61801.

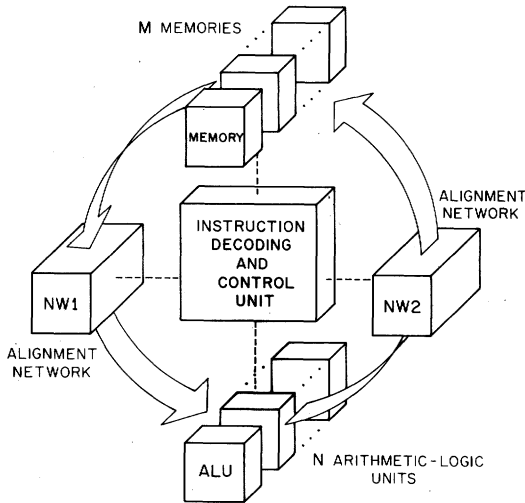


Fig. 1. An array processor.

MEMORY			
0	1	2	3
a_{00}	a_{01}	a_{02}	a_{03}
a_{13}	a_{10}	a_{11}	a_{12}
a_{22}	a_{23}	a_{20}	a_{21}
a_{31}	a_{32}	a_{33}	a_{30}

Fig. 2. 1-skew storage.

which used data skewing and a prime number of memories M , where $M > N = 2^n$. Their scheme allowed conflict-free access of rows, columns, diagonals, backward diagonals, and $N^{1/2} \times N^{1/2}$ blocks of $N \times N$ arrays.¹ However, their scheme has two drawbacks. First, address generation is difficult, involving modulo M arithmetic. Second, the only known data switch capable of performing the necessary alignment of data from a prime number of input ports is a full crossbar switch. (Alignment requirements will be discussed in more detail in the next section.) But for large N , crossbar switches are generally too expensive.

In this section we will present a new storage scheme which allows conflict-free access to rows, columns, forward and backward diagonals, and square blocks. In a later section we will present a data switching network which has fewer gates than a crossbar switch but which can still provide the necessary data alignment functions. We begin by generalizing the concept of skewed storage.

A. Generalized Skewing and Conflict-Free Access

Let M be the number of memories and let δ_i be the skewing distance in the i th dimension. Thus, for a two-dimensional ($N \times N$) matrix, element a_{jk} will be stored in memory $j \cdot \delta_1 + k \cdot \delta_2$, i.e., each successive element of the first dimension (column) is stored δ_1 (mod M) memories away from the previous element. Similarly for the second dimension (rows) and δ_2 . This is called a (δ_1, δ_2) skewing scheme. Fig. 3 illustrates a system with $M = 5$, $\delta_1 = 2$,

MEMORY				
0	1	2	3	4
a_{00}	a_{02}	$\times$	a_{01}	a_{03}
a_{11}	a_{13}	a_{10}	a_{12}	$\times$
a_{22}	$\times$	a_{21}	a_{23}	a_{20}
a_{33}	a_{30}	a_{32}	$\times$	a_{31}

Fig. 3. Five memories with $\delta_1 = 2$ and $\delta_2 = 3$.

and $\delta_2 = 3$. Clearly this generalizes to matrices with k dimensions (i.e., $(\delta_1, \delta_2, \dots, \delta_k)$ skewing) and matrices whose dimensions are larger than N .

Define a d -ordered N -vector (mod M) as a vector of N elements whose i th logical element is stored in memory unit $di + c$ (mod M) where c is an arbitrary constant. For example in Fig. 3, the third row, $a_{22}, \times, a_{21}, a_{23}, a_{20}$ is a 3-ordered 4-vector (mod 5) since the i th element is in memory μ where $\mu = 3i + 4$ (mod 5), $0 \leq i \leq 3$.

A sufficient condition for a d -ordered N -vector (mod M) to be accessible without conflicts is for the following condition to hold:

$$M \geq N \gcd(d, M) \quad (1)$$

where $\gcd(d, M)$ is the greatest common divisor of d and M . This follows from the fact that the set $\{\mu \mid \mu = di + c \pmod{M}, 0 \leq i \leq N-1\}$ must contain N distinct elements. That is, the memories in which the N elements of the d -ordered N -vector are stored must be distinct. This will be true if Condition (1) is met.

If we use a (δ_1, δ_2) skewing scheme, then clearly columns will be δ_1 -ordered, and rows will be δ_2 -ordered. Similarly, diagonals will be $\delta_1 + \delta_2$ -ordered. Thus, in order to access these three types of N -vectors the following conditions must hold:

$$\begin{aligned} M &\geq N \gcd(\delta_1, M) && \text{(columns)} \\ M &\geq N \gcd(\delta_2, M) && \text{(rows)} \\ M &\geq N \gcd(\delta_1 + \delta_2, M) && \text{(forward diagonals)} \\ M &\geq N \gcd(\delta_1 - \delta_2, M) && \text{(backward diagonals).} \end{aligned}$$

Clearly, if $M = N$ then $\gcd(\delta_1, M)$, $\gcd(\delta_2, M)$, and $\gcd(\delta_1 + \delta_2, M)$ must equal 1 if these conditions are to hold. If M is even then δ_1 , δ_2 , and $\delta_1 + \delta_2$ must be odd for this to hold. But it is easy to show that δ_1 , δ_2 , and $\delta_1 + \delta_2$ cannot all be odd. Thus, we cannot have conflict-free access to rows, columns, and diagonals if $M = N$ and M is even.

Let us choose $M = 2N$ memories where N is the number of processors, N is an even power of two, and use $(N^{1/2} + 1, 2)$ skewing. This is illustrated in Fig. 4 for $N = 4$. (Notice that storage is somewhat "sparse." However, another matrix can be stored in the "holes" created by this matrix or this matrix can be squeezed into half as many rows.)

Using this scheme we get the following results.

- 1) Rows are 2-ordered.
- 2) Columns are $(N^{1/2} + 1)$ -ordered.
- 3) Diagonals are $(N^{1/2} + 3)$ -ordered.
- 4) Backward diagonals are $(N^{1/2} - 1)$ -ordered.

It is easy to see that $N^{1/2} + 1$, $N^{1/2} + 3$, and $N^{1/2} - 1$

¹ The need for backward diagonals, e.g., $a_{03}, a_{12}, a_{21}, a_{30}$, comes from the need to sweep certain meshes at a 45° angle (see Muraoka [8] and Kuck and Chen [9]). Square blocks are used to improve the "resolution" of the array processor (see Kuck [4]), as well as in certain important computations (see Kuck and Chen [9]).

MEMORY							
0	1	2	3	4	5	6	7
a_{00}		a_{01}		a_{02}		a_{03}	
	a_{13}		a_{10}		a_{11}		a_{12}
a_{21}		a_{22}		a_{23}		a_{20}	
	a_{30}		a_{31}		a_{32}		a_{33}

Fig. 4. A memory system with $M = 2N = 8$, using $(N^{1/2} + 1, 2)$ skewing.

are all odd and therefore are prime to M which is a power of two. Thus, $\gcd(d, M) = 1$ and the condition $M \geq N \cdot \gcd(d, M)$ is satisfied for columns, diagonals, and backward diagonals. For rows we see that

$$d_{\text{row}} = 2 \quad \text{and} \quad \gcd(d_{\text{row}}, M) = 2,$$

so the condition is again satisfied ($M \geq 2N$). Thus, we have conflict-free memory access to these N -vectors. $N^{1/2} \times N^{1/2}$ blocks are not well ordered, but nevertheless it can be shown that these can be accessed without conflict.

Theorem 1: The elements of a $N^{1/2} \times N^{1/2}$ square block lie in distinct memory modules when stored $(N^{1/2} + 1, 2)$ skewed in $2N$ memories.

Proof: Since we are using $(N^{1/2} + 1, 2)$ skewing, elements of the $N^{1/2} \times N^{1/2}$ block $a_{i,j}, a_{i,j+1}, \dots, a_{i,j+N^{1/2}-1}, a_{i+1,j}, \dots, a_{i+1,j+N^{1/2}-1}, \dots, a_{i+N^{1/2}-1,j}, \dots, a_{i+N^{1/2}-1,j+N^{1/2}-1}$ will be stored in memories numbered $\mu_{x_1, x_2} = (i + x_1) \cdot (N^{1/2} + 1) + (j + x_2) \cdot 2 \pmod{2N}$. For $0 \leq x_1, x_2 \leq N^{1/2} - 1$ we need only show that these memories are distinct, i.e., that² $\mu_{x_1, x_2} \not\equiv_{2N} \mu_{y_1, y_2}$ unless $x_1 = y_1$ and $x_2 = y_2$. Assume $\mu_{x_1, x_2} \equiv_{2N} \mu_{y_1, y_2}$. Then we will show that $x_1 = y_1$ and $x_2 = y_2$. Eliminate the i and j terms and we get

$$x_1(N^{1/2} + 1) + 2x_2 \equiv_{2N} y_1(N^{1/2} + 1) + 2y_2.$$

Expanding and regrouping terms we get

$$N^{1/2}(x_1 - y_1) + x_1 - y_1 \equiv_{2N} 2(y_2 - x_2). \quad (2)$$

Without loss of generality, assume $x_1 > y_1$ so that

$$(x_1 - y_1) < N^{1/2}.$$

Then we get

$$x_1 - y_1 \equiv_{N^{1/2}} 2(y_2 - x_2). \quad (3)$$

Also from (2) since $(x_1 - y_1) < N^{1/2}$, we get

$$(x_1 - y_1) \equiv_{N^{1/2}} 2(y_2 - x_2) \div N^{1/2}. \quad (4)$$

From (3) and (4) we get

$$2(y_2 - x_2) \equiv_{N^{1/2}} 2(y_2 - x_2) \div N^{1/2}. \quad (5)$$

Let

$$2(y_2 - x_2) \equiv_{2N} aN^{1/2} + b, \quad 0 \leq a, b \leq N^{1/2} - 1;$$

² The notation $a \equiv_c b$ means a is congruent to b modulo c .

then from (5) we get

$$aN^{1/2} + b \equiv_{N^{1/2}} a$$

so that $a, b = 0$, i.e., $2(y_2 - x_2) \equiv_{2N} 0$ and since $x_2, y_2 < N^{1/2}$, $x_2 = y_2$. Using this and (2) again we get

$$N^{1/2}(x_1 - y_1) + (x_1 - y_1) \equiv_{2N} 0.$$

So since $x_1, y_1 < N^{1/2}$, $x_1 = y_1$.

Q.E.D.

Thus, we have shown that using $2N$ memories where N is an even power of 2 with a $(N^{1/2} + 1, 2)$ skewing scheme we can provide conflict-free access to rows, columns, forward and backward diagonals, and $N^{1/2} \times N^{1/2}$ blocks. Of course this solution is not unique. But as we shall show, the data alignments required with this solution can be provided by a special switching network whose time-gate product is more attractive than a crossbar switch.

B. Alignment Requirements

Up to this point we have described various types of N -vectors which we would like to access. Now we will discuss what we want to do with these N -vectors and describe the required alignments of these N -vectors. By *alignment* of an N -vector we mean taking the N elements and placing them in some specified order. This function is performed by the networks shown in Fig. 1.

As described earlier, (δ_1, δ_2) skewing results in N -vectors which are not in logical order. Given any two N -vectors, in order to align these for operations between corresponding pairs of elements we must first shift one or both vectors so that their first elements are in the same physical location (processor). Then, if their ordering is not the same we must change the ordering of one of the vectors to the ordering of the other, or change the ordering of both vectors to a common third ordering. After the operation is performed we may have to shift and change the ordering of the vector in order to store the result appropriately. To simplify matters in this paper, we shall assume that we will always change d -ordered vectors to 1-order for processing and vice versa for storage of the result.

If we allow operations between any pair of N -vectors corresponding to rows, columns, or diagonals, then our alignment requirements thus far are that we be able to shift arbitrary distances and at the same time change d -ordered vectors to 1-ordered vectors for $d = \delta_1, \delta_2, \delta_1 + \delta_2, \delta_1 - \delta_2$, or change 1-ordered vectors to the above d -ordered vectors.

Several requirements for alignment of square blocks can also be identified. Two such requirements are to place the blocks in row major or column major order. Two other requirements can also be identified. Consider taking an $N^{1/2}$ -vector and *expanding* it to an N -vector in two special ways as shown in Fig. 5. In one case the $N^{1/2}$ -vector is duplicated $N^{1/2}$ times to make an N -vector. In the other case each element of the $N^{1/2}$ -vector is fanned out $N^{1/2}$ times to make an N -vector.

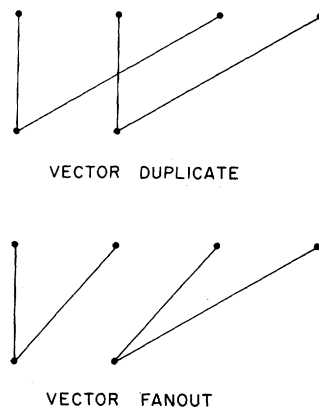


Fig. 5. Vector duplicate and vector fanout with $N = 4$.

If the $N^{1/2}$ -vector which is being expanded consists of consecutive elements of a row, then we call this *row fanout* or *row duplication*. Similarly for columns, etc. These patterns are useful, for example, when multiplying two $N^{1/2} \times N^{1/2}$ blocks using N processors in $N^{1/2}$ steps.

Thus far we have described a variety of alignment requirements. However, we make no claim that these requirements are either necessary or sufficient. We only believe that these patterns account for a vast majority of cases which would be required for an array processor intended for numerical applications. Verification of this will have to await empirical evaluation of real programs.

III. ALIGNMENT NETWORKS

As we have seen, array processing requires the capability for some complex data alignments. In order to provide these capabilities we must build some complex switches. To some it may seem unusual and unnecessary to provide expensive and complex pieces of hardware just to channel the data from one place to another. Yet when one considers the cost and complexity of the memory exchanges, buses, ports with priority resolution, etc., which are used in more conventional computer systems, this should not seem so unusual. When one further observes expensive memories and processors idling while waiting for data alignments, more expensive alignment hardware should not appear unnecessary.

In this section we will discuss some switches or *alignment networks* and we will present a network which provides the capabilities discussed in the previous section.

The traditional $N \times M$ crossbar switch is one possibility for an alignment network. This switch can perform any one-to-one mapping of inputs to outputs and with slight modification, can do any one-to-many mapping. The time required to do this mapping is³ $O(\log N)$ or $O(\log M)$ gate delays. Unfortunately, the number of gates in an $N \times M$ crossbar is proportional to $N \cdot M$, and for large systems this may be overly expensive in terms of gates and reliability.

Another possibility is the rearrangeable switching net-

work discussed by Benes [10]. This network has the same capability as a crossbar but only $O(N \log N)$ gates for an $N \times N$ network. The time to pass through the network is $O(\log N)$. Unfortunately, it is not easy to set up this network to do a particular alignment. The best known algorithm for doing this (Opferman and Tso Wu [11]) requires $O(N \log N)$ time units. This is too long to be practical in this application.

A third possibility is the Batcher sorting network [12]. This network also has the capabilities of a crossbar switch. It requires $O(N(\log N)^2)$ gates, $O(\log N)^2$ time units to pass through the network, and can be set up "on the fly" (i.e., at approximately the same time the data are passing through the network). This network is appealing because it is cheaper than the crossbar and faster than the Benes network. Other networks proposed by Waksman [13], Thurber [14], Kautz [15], and Rohrbacher [16] might also be adapted to meet our needs. But as we shall see, another possibility exists which is relatively cheap and fast, and even though it does not have the full capabilities of a crossbar switch, it is capable of performing the desired alignment functions.

We shall now examine the construction and properties of the "omega" network. An example of an 8×8 omega network is shown in Fig. 6. For the present, we shall only consider $N \times N$ networks where N is a power of two. Other sizes can be constructed by building the next larger power of two and deleting unnecessary switching elements and interconnections, or by using a nonbinary omega network (see Lawrie [17]).

A. Construction of an Omega Network

An $N \times N$ omega network consists of $l = \log N$ identical stages. Each stage consists of a *perfect shuffle*⁴ interconnection followed by $N/2$ switching elements as shown in Fig. 6.

Each switching element can have one of the four states shown in Fig. 7. That is, it may either send its inputs *straight through*, *interchange* the inputs, or *broadcast* one of the inputs to both outputs. We do not allow both inputs to be switched to the same output.

The perfect shuffle connection has the property of taking an input at a position whose binary representation is $s_1s_2 \dots s_l$, and moving it to position $s_2s_3 \dots s_l s_1$. The switch can then move the output to $s_2s_3 \dots s_l 0$ or $s_2s_3 \dots s_l 1$.

In order to switch data through the network, each element in the network is set in one of the above four states (not necessarily all the same) and then the data are allowed to pass from the network inputs to the network outputs.

⁴ The perfect shuffle connection is known to have certain special properties as described by Golomb [18], Pease [19], and Stone [1], among others. In fact, other important switching networks described by Benes [10] and Batcher [12] can be shown to consist of various stages interconnected by this same perfect shuffle. The number of stages in the omega network ($\log N$) is one of the primary differences between the omega network, the Batcher sorting network ($\log N(\log N + 1)/2$ stages) and the binary Benes rearrangeable network ($2(\log N) - 1$ stages). The logic of each switching element is also somewhat different in these three types of networks, but of approximately the same complexity.

³ The notation $O(\log N)$ means "order of $\log N$." All logarithms in this paper are base 2.

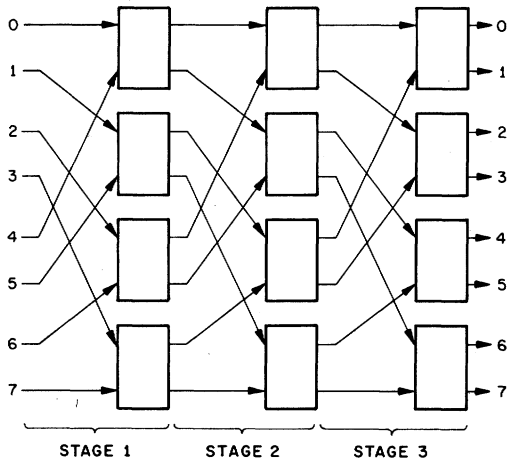
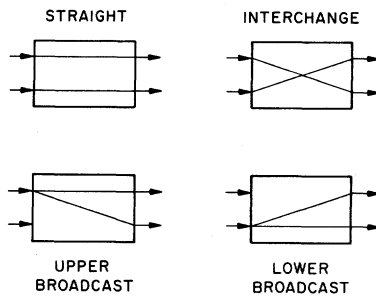
Fig. 6. An 8×8 omega network.

Fig. 7. Four allowed states of the switching elements shown in Fig. 6.

This affects a one-to-one or one-to-many mapping of inputs to outputs.

Two questions arise at this point. First, is there an algorithm which determines the necessary states for the switching elements and can this be done in the same order of time as it takes data to pass through the network? Second, what kinds of input-output mappings are possible? This latter question will be discussed shortly. The first question concerns the existence of a rapid control algorithm for the network.

B. Controlling the Omega Network

As it turns out, there is an efficient algorithm for setting the states of the omega network. First, consider switching input number S to output number D . By examining Fig. 6, it should be easy to see that there is one and only one path between any given input, output pair. Let $D = d_1d_2 \dots d_l$ be the *destination tag*, i.e., the binary representation of the output number to which input number S is to be connected, and let $S = s_1s_2 \dots s_l$ be the *source tag*, i.e., the binary representation of the input number. Starting at input S , the first switch to which S is connected is set to switch input S to the upper output if $d_1 = 0$ or the lower output if $d_1 = 1$. This is shown in Fig. 8 for $S = 010$, $D = 110$. Following this through to a switch in the next stage, we again switch the input to the upper output if $d_2 = 0$ or to the lower output if $d_2 = 1$. We continue in this manner, switching on d_i at each stage i , until we get to the proper output. It is easy to see that during any

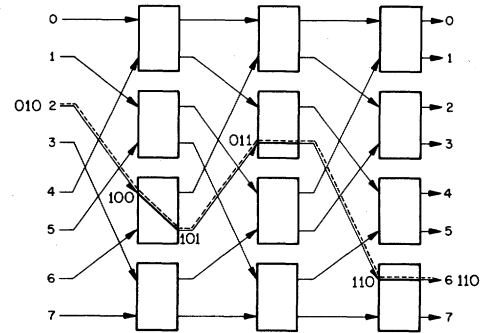


Fig. 8. The connection (010,110).

given stage i , an input which has been switched to position $s_i s_{i+1} \dots s_l d_1 d_2 \dots d_{i-1}$ goes through the perfect shuffle and ends up in position $s_{i+1} s_{i+2} \dots s_l d_1 d_2 \dots d_{i-1} s_i$, and is then switched into position $s_{i+1} s_{i+2} \dots s_l d_1 d_2 \dots d_{i-1} d_i$. Thus, after $l = \log N$ operations, the original input must be connected to output $d_1 d_2 \dots d_l$. A similar algorithm works by starting at output $d_1 d_2 \dots d_l$ and working backwards through the network, switching the element at stage i according to s_i .

In order to set up a particular mapping of inputs to outputs, we simply follow the above procedure simultaneously for all inputs or all outputs. It remains for us to show that this procedure is complete in the sense that it can set up any mapping of which the network is capable. We will not prove this formally (see Lawrie [17]). Instead, notice that the algorithm always chooses one path through the network between any given input and output. Thus the algorithm will specify paths for any mapping of inputs to outputs. Since there is one and only one path between any input and output, the set of paths for a given mapping must be unique, and this must be the set determined by the algorithm. Notice, however, that the algorithm can specify path sets for mappings of which the network is not capable. For example, Fig. 9 shows the paths established for the mapping $000 \rightarrow 000$, $100 \rightarrow 010$. These paths share a common connection at the output of the first stage—a condition which we call a conflict and which by definition is not allowed since it means two different signals must share a common wire. (We assume two signals can share a common wire only if both signals have a common source, in which case we assume they are identical signals.) Thus we see that the algorithm will choose the unique set of paths in the network for any given mapping, but the paths in a set will not necessarily be disjoint, i.e., a conflict may be present.

There are many options available when it comes to actually designing an omega network. We will briefly mention some of them here. Further details can be found in Lawrie [17]. The switching elements in Fig. 6 are 2×2 exchange-broadcast units. It is possible to construct an omega network from larger elements as illustrated in Fig. 10. Here an 8×8 network is constructed from 4×4 and 2×2 elements. The 4×4 elements could be cross-bar/broadcast elements in which case this network is somewhat more powerful than the network constructed

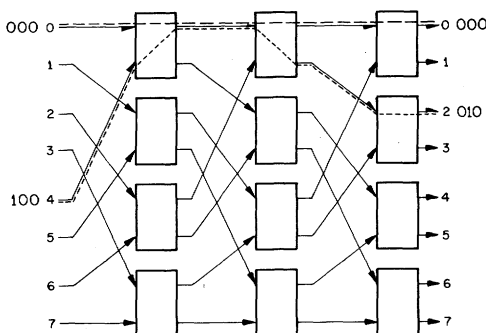
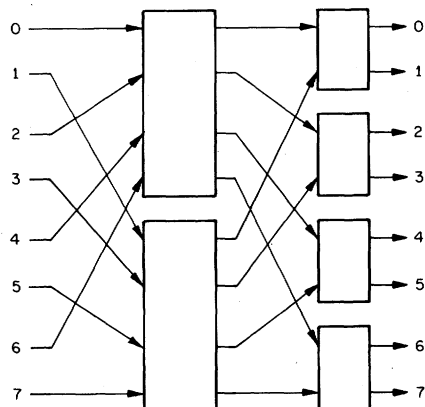


Fig. 9. Paths for (000,000), (100,010).

Fig. 10. An omega network using 4×4 and 2×2 elements.

solely from 2×2 elements. Or these 4×4 elements could be 4×4 omega networks constructed from 2×2 elements, in which case this network is identical to one constructed solely from 2×2 elements. Thus, the omega network can be *partitioned* to suit the architecture of a machine and the packaging requirements of a given circuit technology.

For example, we could connect a group of 16 processors with 16 memories using a 16×16 crossbar/broadcast switch, and then interconnect four such groups with two shuffle, exchange stages. This would allow complex data mapping capabilities within groups of 16 processors and provide more restricted omega-type alignments between all 64 processors.

Another option involves building only one stage of an omega network. Suppose we build a $2^l \times 2^l$ omega network from elements of a uniform size $K = 2^k$ where l is an integer multiple of k , i.e., $\sigma k = l$. Then this network will have σ stages, and each stage will be identical. It is not necessary to construct all σ stages. We only need to build one such stage and recycle this stage σ times. Of course this requires some extra registers, etc., but this solution may save a lot of gates, and it gives us the possibility of recycling other than exactly σ times which allows (us to perform some mappings which cannot be done in exactly σ stages (cf. Lang [20] and Lawrie [17])).

C. Properties of Omega Networks

It should be clear from the last section that due to conflicts, an omega network cannot produce all possible one-

to-one or one-to-many mappings of inputs to outputs. In this section we will demonstrate classes of mappings which can be produced.

Definition: Let $a = a'm + \alpha$ and $b = b'm + \beta$ where $\alpha, \beta < m$ and $a, b, \alpha, \beta, a', b'$, and m are all nonnegative integers. We say $a \equiv_m b$ if and only if $\alpha = \beta$. This is the common definition of "a is congruent to b modulo m."

Definition: Assume m is a factor of N . With a and b as above, we say $a \equiv_N^m b$ if and only if $a'm \equiv_N b'm$, i.e., $m[a/m] \bmod N = m[b/m] \bmod N$.

For example, let $a = 11010$ (binary), $b = 01010$, and $c = 10010$. Then $a \equiv_4 b \equiv_4 c$, $a \equiv_{16}^2 b$ (since $101 = 101$) and $a \not\equiv_{16}^2 c$ (since $101 \neq 001$).

Shortly we will present some proofs which will be easier to follow if at this point we present, without proof, some results from number theory. In the following, " $\rightarrow$ " means logical implication and " $\leftrightarrow$ " means logical equivalence.

$$R1) \quad x + a \equiv_m y + a \leftrightarrow x \equiv_m y.$$

$$R2) \quad x \equiv_m y \rightarrow ax \equiv_m ay.$$

$$R3) \quad \text{If } a \text{ is prime to } m \text{ (i.e., } \gcd(a, m) = 1\text{), then}$$

$$ax \equiv_m ay \rightarrow x \equiv_m y.$$

$$R4) \quad ax \equiv_m ay \leftrightarrow x \equiv_m y.$$

$$R5) \quad \text{If } 0 \leq x, y < m, \text{ then}$$

$$x \equiv_m y \leftrightarrow x \equiv_m y.$$

$$R6) \quad x \equiv_m y \text{ and } x \not\equiv_N y \rightarrow x \not\equiv_N y.$$

Definition: Define P_N to be a set of integer pairs $P_N = \{(S_i, D_i) \mid 0 \leq i < N\}$ which represents a mapping of inputs to outputs.

$$S_0 \rightarrow D_0, S_1 \rightarrow D_1, \dots, S_q \rightarrow D_q, \quad q < N.$$

Definition: Let Ω_N be the $N \times N$ omega network as described above,⁵ where $N = 2^l$. If Ω_N can produce the mapping specified by P_N then we say Ω_N passes P_N or $\Omega_N \uparrow P_N$. Otherwise, we say $\Omega_N \nmid P_N$.

Recall from the previous section, that given an input-output connection represented by $(s_1 s_2 \dots s_l, d_1 d_2 \dots d_l)$, after k stages input $s_1 s_2 \dots s_l$ will be connected to output $s_{k+1} s_{k+2} \dots s_l d_1 d_2 \dots d_k$ of stage k . Given a set

$$P_N = \{(S_i, D_i) \mid 0 \leq i < N\}$$

representing an input-output mapping, then $\Omega_N \uparrow P_N$ unless there is a conflict, i.e., unless two distinct inputs are connected to the same output of some stage. That is, for some pair of connections $(S_i, D_i), (S_j, D_j) \in P_N$ and

⁵ In the discussions to follow, we will only concern ourselves with omega networks whose size is a power of two. For networks of other sizes, see Lawrie [17].

for some k , we have $s_{i,k+1}s_{i,k+2}\cdots s_{i,l} = s_{j,k+1}s_{j,k+2}\cdots s_{j,l}$ and $d_{i,1}d_{i,2}\cdots d_{i,k} = d_{j,1}d_{j,2}\cdots d_{j,k}$. This is equivalent to saying that for some $m = 2^{l-k}$, $1 \leq k \leq l$, and for some (S_i, D_i) and (S_j, D_j) , we have $S_i \not\equiv_N S_j$ and $S_i \equiv_m S_j$ and $D_i \equiv_m D_j$. Thus we have the following theorem.

Theorem 2: Given a set of desired input-output connections $P_N = \{(S_i, D_i) \mid 0 \leq i < N\}$, then $\Omega_N \uparrow P_N$ if and only if $S_i \not\equiv_N S_j$ and $S_i \equiv_m S_j$ implies $D_i \not\equiv_m D_j$ for all $S - D$ pairs in P_N and for all $m = 2^k$, where $1 \leq k \leq l$.

Now we will begin to identify some of the mappings which the omega network can produce.

Theorem 3: Let

$$P_N(a, b, c, d, \xi) = \{(ax + b, cx + d) \mid 0 \leq x < \xi\}$$

where $(ax + b, cx + d)$ represents a connection from input $ax + b \pmod{N}$ to output $cx + d \pmod{N}$. If $\gcd(a, N) \leq \gcd(c, N)$ and $\xi \leq N/\gcd(c, N)$ then $\Omega \uparrow P_N(a, b, c, d, \xi)$.

Proof: Let $\alpha = \gcd(a, N)$, $\gamma = \gcd(c, N)$, $a' = a/\alpha$ and $c' = c/\gamma$. We must show that if $\alpha \leq \gamma$ and $\xi \leq N/\gamma$ then Theorem 2 is satisfied, i.e., for all $0 \leq x, y < \xi$ and for all $m = 2^k$ where $1 \leq k \leq l$ and $2^l = N$, then $ax + b \not\equiv_N ay + b$ and $ax + b \equiv_m ay + b$ implies $cx + d \not\equiv_m cy + d$. We begin with the hypothesis $ax + b \equiv_m ay + b$ and by R1 we get $ax \equiv_m ay$ and since a' is prime to m we get $\alpha x \equiv_m \alpha y$ by R3. Now since $(\gamma/\alpha) \geq 1$ and since $c'(\gamma/\alpha)\alpha = c$, by R2 we get

$$cx \equiv_m cy$$

and by R1

$$cx + d \equiv_m cy + d. \quad (6)$$

We also have as a hypothesis that

$$ax + b \not\equiv_N ay + b$$

so by R1, R2 we get

$$x \not\equiv_N y$$

and by R4 we get

$$\gamma x \not\equiv_{\gamma N} \gamma y. \quad (7)$$

But we know $x, y < \xi \leq N/\gamma$ so $\gamma x, \gamma y < N$ and from (7) and R5 we get

$$\gamma x \not\equiv_N \gamma y,$$

and since c' is prime to m , by R3 we get

$$cx \not\equiv_m cy$$

and so

$$cx + d \not\equiv_m cy + d. \quad (8)$$

Now, from (6) and (8) by R6 we get

$$cx + d \not\equiv_N cy + d. \quad \text{Q.E.D.}$$

Corollary 1: Define $I_N = \{(x, x) \mid 0 \leq x \leq N - 1\}$ to be the identity mapping. Then $\Omega_N \uparrow I_N$. This follows trivially by setting $a = c = 1$ and $b = d = 0$ in the previous theorem, and noting that $\alpha = \gamma = 1$ and so $\alpha \leq \gamma$ and $x < \xi \leq N/\gamma$.

Corollary 2: Let $P_N = \{(ax + b, x) \mid a \leq x < N\}$ where a is odd. Then $\alpha = 1$, $\gamma = 1$, and $\xi = N$. So $\Omega_N \uparrow P_N$ since $\alpha \leq \gamma$ and $\xi \leq N/1$. Thus, the omega network can unscramble and shift any a -ordered vector where a is odd.

Theorem 4: If $\Omega_N \uparrow P_N$ and a is odd, then $\Omega_N \uparrow P_N \cdot a$ where $P_N = \{(S_i, D_i) \mid 0 \leq i < N\}$ and $P \cdot a$ is the set $\{(a \cdot S_i \pmod{N}, D_i) \mid (S_i, D_i) \in P_N\}$.

Proof: By contradiction, assume $\Omega_N \uparrow P_N \cdot a$. Then there exists an m and $(aS_i, D_i), (aS_j, D_j) \in P_N \cdot a$ such that $aS_i \not\equiv_N aS_j$, $aS_i \equiv_m aS_j$ and $D_i \equiv_m D_j$. But since a is odd, $\gcd(a, m) = 1$ so $aS_i \equiv_m aS_j \rightarrow S_i \equiv_m S_j$ and thus $\Omega_N \not\uparrow P_N$. Thus, Theorem 2 must be satisfied and so $\Omega_N \uparrow P_N$. Q.E.D.

Theorem 5: If $\Omega_N \uparrow P_N$ then $\Omega_N \not\uparrow P_N + a$ where $P_N = \{(S_i, D_i) \mid 0 \leq i < N\}$, and $P_N + a$ is the set $\{(S_i + a \pmod{N}, D_i) \mid (S_i, D_i) \in P_N\}$.

Proof: By contradiction, assume $\Omega_N \uparrow P_N + a$. Then there exists an m and $(S_i + a, D_i), (S_j + a, D_j) \in P_N + a$ such that $S_i + a \not\equiv_N S_j + a$, $S_i + a \equiv_m S_j + a$, and $D_i \equiv_m D_j$. But then $S_i \not\equiv_N S_j$ and $S_i \equiv_m S_j$ so $\Omega_N \not\uparrow P_N$. So again Theorem 2 must be satisfied. Q.E.D.

The mapping $P_N + a$ is called a *uniform shift* (by a) of P_N . Theorem 5 states that if an omega network can produce the mapping P_N , then it can produce a uniform shift of this mapping.

Theorem 6: Given $P = \{(S_i, D_i)\}$. If $\Omega_N \uparrow P$ then $\Omega_{2N} \uparrow P$.

Proof: Suppose $\Omega_{2N} \not\uparrow P$. Then there exists an m and there exists $(S_i, D_i), (S_j, D_j) \in P$ such that $S_i \equiv_m S_j$, $S_i \not\equiv_{2N} S_j$, and $D_i \equiv_m D_j$. m cannot be $2N$ as this would lead to an immediate contradiction. If $m < 2N$ then $\Omega_N \uparrow P$. Thus, Theorem 6 is true by contradiction. Q.E.D.

Theorem 7: Let $P_N = \{(S_i, D_i) \mid 0 \leq S_i, D_i < N\}$. Define $P_{aN} * a = \{(S_i, a \cdot D_i)\}$. If a is a power of two and if $\Omega_N \uparrow P_N$ then $\Omega_{aN} \uparrow P_{aN} * a$.

Proof: From Theorem 6, $\Omega_N \uparrow P_N \rightarrow \Omega_{aN} \uparrow P_N$. Now $\Omega_{aN} \uparrow P_N$ implies that for all $m = 2, 4, 8, \dots, aN$, if $S_i \not\equiv_{aN} S_j$ and $S_i \equiv_m S_j$ then $D_i \not\equiv_m D_j$. Since $aD_i, aD_j < aN$, $D_i \not\equiv_m D_j$ implies $aD_i \not\equiv_m aD_j$. Q.E.D.

Theorem 8: Let $P_N = \{(S_i, D_i) \mid 0 \leq S_i, D_i < N\}$. Define $P_{aN} \cdot a * a = \{(aS_i, aD_i)\}$ and let a be a power of two. If $\Omega_N \uparrow P_N$ then $\Omega_{aN} \uparrow P_{aN} \cdot a * a$.

Proof: We must show that Theorem 2 is satisfied, i.e., that for all $am = 2, 4, 8, \dots, aN$, $aS_i \not\equiv_{aN} aS_j$ and $aS_i \equiv_{am} aS_j$ imply $aD_i \not\equiv_{am} aD_j$. Assume $aS_i \not\equiv_{aN} aS_j$ and $aS_i \equiv_{am} aS_j$. Then $S_i \not\equiv_N S_j$ and $S_i \equiv_m S_j$ for all $m = 1, 2, 4, \dots, N$. Since $\Omega_N \uparrow P_N$ this implies $D_i \not\equiv_N D_j$. This in turn implies $aD_i \not\equiv_{aN} aD_j$. Q.E.D.

For Theorem 9 we will need the following lemma.

Lemma 1: Let $0 \leq x_1, y_1 \leq n-1$ and $0 \leq x_2, y_2 \leq a-1$. Then $ax_1 + x_2 \equiv_{an} ay_1 + y_2$ implies $x_1 = y_1$ and $x_2 = y_2$.

Proof: Assume $ax_1 + x_2 \equiv_{an} ay_1 + y_2$. Then by R2 $anx_1 + nx_2 \equiv_{an} any_1 + ny_2$. Since $anx_1 \equiv_{an} 0$ we get $nx_2 \equiv_{an} ny_2$. Thus, by R4 $x_2 \equiv_a y_2$. Since $0 \leq x_2, y_2 < a$ we get $x_2 = y_2$ and so $ax_1 \equiv_{an} ay_1$. Thus, by R4, $x_1 \equiv_n y_1$ and since $0 \leq x_1, y_1 < n$ we get $x_1 = y_1$. Q.E.D.

Theorem 9: $\Omega_M \uparrow P(x_1, x_2)$ where $M = 2n^2$, n a power of two, and

$$P(x_1, x_2)$$

$$= \{(2x_1 + (n+1)x_2, 2x_1 + 2nx_2) \mid 0 \leq x_1, x_2 \leq n-1\}.$$

Proof: The proof will be in two parts, first for $m = 2, 4, 8, \dots, 2n$ and then for $m = 4n, 8n, \dots, 2n^2$. In each case we will show that $2x_1 + (n+1)x_2 \equiv_m 2y_1 + (n+1)y_2$ and $2x_1 + 2nx_2 \equiv_m 2y_1 + 2ny_2$ implies $2x_1 + (n+1)x_2 \equiv_m 2y_1 + (n+1)y_2$ thus satisfying Theorem 2.

Part 1: Let $x = x_1 + nx_2$, $y = y_1 + ny_2$, and let $x_1 = am/2 + \alpha$, $y_1 = bm/2 + \beta$ where $\alpha, \beta < m/2$ and assume $m \leq 2n$. If $2x \equiv_m 2y$ then $2nx_2 + am + 2\alpha \equiv_m 2ny_2 + bm + \beta$ and since $\alpha, \beta < m/2$ and $m \leq 2n$, it follows that $2nx_2 + am \equiv_m 2ny_2 + bm$.

Since $am, bm < 2n$ we have by Lemma 1 $am = bm$ or $a = b$, and so $x_2 = y_2$. Now if $2x_1 + (n+1)x_2 \equiv_m 2y_1 + (n+1)y_2$, then since $x_2 = y_2$ we have $2x_1 \equiv_m 2y_1$ and since $a = b$ we get $2\alpha \equiv_m 2\beta$. Now, $\alpha, \beta < m/2$ so we have $\alpha = \beta$. Thus, $2x_1 + (n+1)x_2 \equiv_m 2y_1 + (n+1)y_2$.

Part 2: Now consider the case where $m = 4n, 8n, \dots, 2n^2$. Let $m = 2gn$ where $g = 2, 4, \dots, n$, and let $x_2 = ag + \alpha$, $y_2 = bg + \beta$ where $\alpha, \beta \leq g-1$. Assume $2x \equiv_m 2y$, i.e., $2x_1 + 2nag + 2\alpha n \equiv_m 2y_1 + 2nbg + 2\beta n$. Since $m = 2ng$ and $2(\alpha n + x_1), 2(\beta n + y_1) < 2ng$ we get $am \equiv_m bm$ and so $a = b$. Now if $2x_1 + (n+1)x_2 \equiv_m 2y_1 + (n+1)y_2$ then since $a = b$ we get

$$2x_1 + (n+1)\alpha \equiv_m 2y_1 + (n+1)\beta.$$

Since $\alpha \leq g-1, m = 2gn$, and $x_1 \leq n-1$ we get

$$\begin{aligned} 2x_1 + (n+1)\alpha &\leq 2(n-1) + (n+1)(g-1) \\ &\leq (g+1)n + g - 3, \end{aligned}$$

so that

$$2x_1 + (n+1)\alpha \leq 2gn.$$

Thus, $2x_1 + (n+1)\alpha \equiv_m 2y_1 + (n+1)\beta$ and since $a = b$ we get $2x_1 + (n+1)(\alpha + ag) \equiv_m 2y_1 + (n+1)(\beta + bg)$. Since $\alpha + ag = x_2, \beta + bg = y_2$ we have

$$2x_1 + (n+1)x_2 \equiv_m 2y_1 + (n+1)y_2. \quad \text{Q.E.D.}$$

Due to limitations on space, the following three theorems are presented without proof. Proofs may be found in Lawrie [21].

Theorem 10: $\Omega_{2n^2} \uparrow Q(x_1, x_2)$ where

$$Q(x_1, x_2)$$

$$= \{((n+1)x_1 + 2x_2, 2x_1 + 2nx_2) \mid 0 \leq x_1, x_2 < n-1\}.$$

Theorem 11: $\Omega_{n^2} \uparrow F(x_1, x_2)$ where

$$F(x_1, x_2) = \{(x_1, (x_1 + x_2)) \mid 0 \leq x_1, x_2 < n-1\}.$$

Theorem 12: $\Omega_n \uparrow G(x_1, x_2)$ where

$$G(x_1, x_2) = \{(x_1, x_1 + x_2n) \mid 0 \leq x_1, x_2 \leq n-1\}.$$

Definition: Define an inverse omega network Ω_N' to be an omega network whose inputs and outputs are reversed and whose switching elements have the same four states shown in Fig. 7, as illustrated in Fig. 11. Further, let P_N be any one-to-one mapping in $N \times N$, and define P_N' to be the inverse of this mapping.

Theorem 13: Given P_N is a one-to-one mapping in $N \times N$, then $\Omega_N \uparrow P_N$ if and only if $\Omega_N' \uparrow P_N'$.

The proof of this should be obvious. Thus any one-to-one mapping which can be done by the omega network can be undone by the corresponding inverse omega network.

Thus, we have presented some theorems which prove that certain basic classes of permutations and one-to-many connections can be produced by the omega network. In the next section we will present some alignment connections which are desirable in an array processor. We will then use the theorems of this section to prove that the omega network can produce these connections.

D. Application of the Omega Network to an Array Processor

Let us now consider an array computer with $N = n^2$ processors (n a power of two) connected to $M = 2N$ memories by a $2N \times 2N$ omega network. This corresponds to NW2 in Fig. 1. The memories are connected to the network input ports in a one-to-one fashion, i.e., memory i is connected to port i . Also, processor i is connected to output port $2i$. Thus, the odd numbered output ports will not be used, so the last column of exchange elements could be replaced by simple or elements. A second network, an inverse Ω network, is then connected in a similar fashion between the processors and memories. This network corresponds to NW1 in Fig. 1.

Finally, assume a $(N^{1/2} + 1, 2)$ skewing scheme. It is easy to show that the following data mappings are needed in order to align rows, columns, etc.

$$P(\text{rows}) = \{(2x + b, 2x) \mid 0 \leq x \leq N-1\}.$$

$$P(\text{columns}) = \{(x(n+1) + b, 2x) \mid 0 \leq x \leq N-1\}.$$

$$P(\text{forward diagonals}) = \{(x(n+3) + b, 2x) \mid 0 \leq x \leq N-1\}.$$

$$P(\text{backward diagonals}) = \{(x(n-1) + b, 2x) \mid 0 \leq x \leq N-1\}.$$

For $N^{1/2} \times N^{1/2}$ blocks we need the following alignments:

$$P(\text{row major}) = \{(2x_1 + (n+1)x_2 + b, 2(x_1 + nx_2)) \mid 0 \leq x_1, x_2 \leq n-1\}.$$

$$P(\text{column major}) = \{(x_1(n+1) + 2x_2 + b, 2(x_1 + x_2)) \mid 0 \leq x_1, x_2 \leq n-1\}.$$

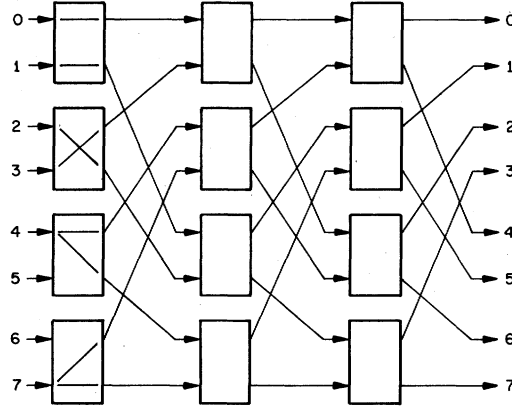


Fig. 11. An 8×8 inverse omega network showing four possible states of the switching elements.

$$P(\text{row fanout}) = \{(2x_1 + b, 2(x_1n + x_2)) \mid 0 \leq x_1, x_2 \leq n - 1\}.$$

$$P(\text{column fanout}) = \{(x_1(n + 1) + b, 2(x_1n + x_2)) \mid 0 \leq x_1, x_2 \leq n - 1\}.$$

$$P(\text{forward diagonal fanout}) = \{(x_1(n + 3) + b, 2(x_1n + x_2)) \mid 0 \leq x_1, x_2 \leq n - 1\}.$$

$$P(\text{backward diagonal fanout}) = \{(x_1(n - 1) + b, 2(x_1n + x_2)) \mid 0 \leq x_1, x_2 \leq n - 1\}.$$

$$P(\text{row duplicate}) = \{(2x_1 + b, 2(x_1 + x_2n)) \mid 0 \leq x_1, x_2 \leq n - 1\}.$$

$$P(\text{column duplicate}) = \{(x_1(n + 1) + b, 2(x_1 + x_2n)) \mid 0 \leq x_1, x_2 \leq n - 1\}.$$

$$P(\text{forward diagonal duplicate}) = \{(x_1(n + 3) + b, 2(x_1 + x_2n)) \mid 0 \leq x_1, x_2 \leq n - 1\}.$$

$$P(\text{backward diagonal duplicate}) = \{(x_1(n - 1) + b, 2(x_1 + x_2n)) \mid 0 \leq x_1, x_2 \leq n - 1\}.$$

We will now show that the omega network can produce these alignments.

Theorem 14: $\Omega_{2N} \uparrow P(\text{rows})$.

Proof: By Theorem 3 where α and $\gamma = 2$ so $\alpha = \gamma$ and $\xi = N$ so $\xi \leq 2N/\gamma$ is satisfied.

Theorem 15: $\Omega_{2N} \uparrow P(\text{columns})$.

Proof: By Theorem 3, since $n + 1$ is prime to 2, we have $\alpha = 1$, $\alpha' = (n + 1)$, $\gamma = 2$, and $\xi = N$. Thus, the conditions are met.

Theorem 16: $\Omega_{2N} \uparrow P(\text{forward diagonals})$.

Proof: As in Theorem 15, since $n + 3$ is prime to two.

Theorem 17: $\Omega_{2N} \uparrow P(\text{backward diagonals})$.

Proof: Again by Theorem 3 since $(n - 1)$ is prime to two.

Theorem 18: $\Omega_{2N} \uparrow P(\text{row major})$.

Proof: Directly from Theorems 5 and 9.

Theorem 19: $\Omega_{2N} \uparrow P(\text{column major})$.

Proof: Directly from Theorems 5 and 10.

Theorem 20: $\Omega_{2N} \uparrow P(\text{row fanout})$.

Proof: Refer to Theorem 11 and note that $P(\text{row fanout}) = F(x_1, x_2) \cdot 2 \cdot 2 + b$. Thus, by Theorem 11 $\Omega_N \uparrow F(x_1, x_2)$ and by Theorems 5 and 8 $\Omega_{2N} \uparrow F(x_1, x_2) \cdot 2 \cdot 2 + b$.

Theorem 21: $\Omega_{2N} \uparrow P(\text{column fanout})$.

Proof: Note that $P(\text{column fanout}) = F(x_1, x_2) \cdot (n + 1) \cdot 2 + b$. Thus the result follows from Theorems 4, 5, 7, and 11.

Theorem 22: $\Omega_{2N} \uparrow P(\text{forward diagonal fanout})$.

Proof: $P(\text{forward diagonal fanout}) = F(x_1, x_2) \cdot (n + 3) \cdot 2 + b$. Again the result follows from Theorems 4, 5, 7, and 11.

Theorem 23: $\Omega_{2N} \uparrow P(\text{backward diagonal fanout})$.

Proof: As before by Theorems 4, 5, 7, and 11 since $P(\text{backward diagonal fanout}) = F(x_1, x_2) \cdot (n - 1) \cdot 2 + b$.

Theorem 24: $\Omega_{2N} \uparrow P(\text{row duplicate})$.

Proof: Refer to Theorem 12 and note that $P(\text{row duplicate}) = G(x_1, x_2) \cdot 2 \cdot 2 + b$. Thus, our result follows from Theorems 5, 8, and 12.

Theorem 25: $\Omega_{2N} \uparrow P(\text{column duplicate})$.

Proof: $P(\text{column duplicate}) = G(x_1, x_2) \cdot (n + 1) \cdot 2 + b$. So the result follows from Theorems 4, 5, 7, and 12.

Theorem 26: $\Omega_{2N} \uparrow P(\text{forward diagonal duplicate})$.

Proof: $P(\text{forward diagonal duplicate}) = G(x_1, x_2) \cdot (n + 3) \cdot 2 + b$. Result follows from Theorems 4, 5, 7, and 12.

Theorem 27: $\Omega_{2N} \uparrow P(\text{backward diagonal duplicate})$.

Proof: $P(\text{backward diagonal duplicate}) = G(x_1, x_2) \cdot (n - 1) \cdot 2 + b$. Result follows from Theorems 4, 5, 7, and 12.

Thus, in Theorems 14–27 we have shown that in addition to being able to produce uniform shifts (Theorem 5) and unscrambling of p -ordered vectors where p is odd (Corollary 2), the Ω network can produce the alignments listed at the beginning of Section III-D.

IV. CONCLUSION

In order for an array process to work efficiently, the ALU's must have access to a steady stream of operand pairs. This implies the need for a large memory system capable of providing parallel access to various slices of the data, i.e., N -vectors. We have shown that in order to provide conflict-free memory access to more than a few trivial types of N -vectors, we must use more than N in-

dependent memories and/or choose a number of memories which are relatively prime to certain parameters of the data storage scheme and the memory access requirements. We have also discussed the need for some special hardware capable of placing (aligning) the data in some specific order for processing. We reviewed some networks from the literature which could provide this alignment, and we presented the omega network whose properties are shown to be particularly useful in this context. Finally, we presented a design for an array processor which uses this omega network and which provides for conflict-free memory access and alignment of rows, columns, diagonals, backward diagonals, and $N^{1/2} \times N^{1/2}$ partitions in either row or column major order, and in addition can produce $N^{1/2}$ -vector fanout and duplication functions.

Another important consideration which must be mentioned concerns generating index values both for the memories and for the alignment networks. Generally, access to an N -vector in memory requires a vector of addresses specifying the address of each element of the N -vector in its memory (the *index vector*) and a similar vector of source or destination tags which will control the alignment network (the *tag vector*). For example, assume each memory in Fig. 4 has its own address register. Loading these registers with the values of 0,3,2,1,0,3,2,1, respectively, would cause access to column 0 (as well as column 2; if column 0 was desired, the alignment network would select for transmission only element of column 0). Adding 2 (mod 4) to this index vector would give us 2,1,0,3,2,1,0,3 which would provide access to column 1 (and column 3). Similarly, tag vectors are constructed to control the alignment networks. Obviously, it is desirable not to have to specify complete index and tag vectors for each memory access. Instead we would like to be able to generate a new index and tag vector from the previous index/tag vector, as for example by addition of a constant. In the case above, index vectors for successive columns were formed by adding 2 (mod 4). (A similar rule holds for tag vectors.) In general, for the N -processor, $2N$ -memory system described in the last section, index/tag vectors can be generated by addition of a constant (modulo a power of two) to a *base* index/tag vector for each type of required N -vector.

However, for systems using M memories where M is relatively prime to the ordering of the desired N -vectors (cf. Section II and Budnik and Kuck [7]), generation of index/tag vectors requires addition modulo M where M is not a power of two. This requirement for "relatively prime modulus" arithmetic is one of the primary drawbacks to such a memory system. (The other drawback is the apparent lack of a cheap alignment network with the necessary capabilities.) For further discussion of index/tag vectors and their generation, see Lawrie [22].

Finally, we have not discussed the problems caused by the inability to keep the entire data set in primary memory. Large data sets stored on a secondary memory device (drum or disk) are typically stored by row or column and primary memory may only be large enough to hold a few

such rows or columns. If a large matrix is stored on a disk or drum by rows and we attempt to access it by columns, we must effectively access the entire matrix in secondary memory and will spend a great deal of time waiting for disk or drum access and we will transfer many words which we will never use. This inefficiency may completely offset any improvement caused by the special primary memory organizations and alignment hardware discussed in this paper. There are two possibilities for coping with this problem. First, we might transpose the matrix when necessary. See Schumann [23] and Eklundh [24] for a further discussion of transposing large matrices. (Some form of alignment network would be particularly useful in doing the necessary row or column permutations called for in the transpose algorithms.) Second, we might take advantage of the fact that many computations, (e.g., matrix multiplication) have the property that their data can be partitioned into square blocks and the computation can be carried out on each square partition in turn. Thus, instead of storing a large matrix on disk by row or column, we might store it by square partition instead. (See also Gold and Kuck [24].) Access to various slices of these partitions then proceeds as described earlier.

ACKNOWLEDGMENT

The author remains indebted to D. Kuck for many discussions and ideas on this subject.

REFERENCES

- [1] H. S. Stone, "Parallel processing with the perfect shuffle," *IEEE Trans. Comput.*, vol. C-20, pp. 153-161, Feb. 1971.
- [2] K. E. Batchner, "STARAN parallel processor system hardware," in *1974 Fall Joint Comput. Conf., AFIPS Conf. Proc.*, vol. 43. Montvale, N. J.: AFIPS Press, 1974, p. 405.
- [3] L. C. Higbie, "The OMEN computers: Associative array processors," in *Dig. Papers, Compcon*, p. 287, 1972.
- [4] D. J. Kuck, "ILLIAC IV software and application programming," *IEEE Trans. Comput.*, vol. C-17, pp. 758-770, Aug. 1968.
- [5] P. W. Kraska, "Array storage allocation," *Dep. Comput. Sci., Univ. Illinois, Urbana-Champaign, Rep. 344*, 1969.
- [6] R. E. Millstein, "Control structures in Illiac IV Fortran," *Commun. Ass. Comput. Mach.*, vol. 16, pp. 621-627, Oct. 1973.
- [7] P. Budnik and D. J. Kuck, "The organization and use of parallel memories," *IEEE Trans. Comput.* (Short Notes), vol. C-20, pp. 1566-1569, Dec. 1971.
- [8] Y. Muraoka, "Parallelism exposure and exploitation in programs," Ph.D. dissertation, Univ. Illinois, Urbana-Champaign, *Dep. Comput. Sci., Rep. 424*, Feb. 1971.
- [9] S.-C. Chen and D. J. Kuck, "Time and parallel processor bounds for linear recurrence systems," *IEEE Trans. Comput.*, vol. C-24, pp. 701-717, July 1975.
- [10] V. E. Benes, *Mathematical Theory of Connecting Networks and Telephone Traffic*. New York: Academic, 1965.
- [11] D. C. Opferman and N. T. Tsao-Wu, "On a class of rearrangeable switching networks," *Bell Syst. Tech. J.*, vol. 50, pp. 1579-1618, May-June 1971.
- [12] K. E. Batchner, "Sorting networks and their applications," in *1968 Spring Joint Comput. Conf., AFIPS Conf. Proc.*, vol. 32. Washington, D. C.: Thompson, 1968, pp. 307-314.
- [13] A. Waksman, "A permutation network," *J. Ass. Comput. Mach.*, vol. 15, pp. 159-163, Jan. 1968.
- [14] K. J. Thurber, "Programmable indexing networks," in *1970 Spring Joint Comput. Conf., AFIPS Conf. Proc.*, vol. 36. Montvale, N. J.: AFIPS Press, 1970, pp. 51-58.
- [15] W. H. Kauntz, K. N. Levitt, and A. Waksman, "Cellular interconnection arrays," *IEEE Trans. Comput.*, vol. C-17, pp. 443-451, May 1968.
- [16] D. L. Rohrbacher, "Advanced computer organization study: Volume I—Basic report; Volume II—Appendices," Air Force Contract AF 30(602)-3550, Apr. 1966, Papers AD 631870 and AD 631871.

- [17] D. H. Lawrie, "Memory-processor connection networks," Univ. Illinois, Urbana-Champaign, Dep. Comput. Sci., Rep. 557, Feb. 1973.
- [18] S. W. Golomb, "Permutation by cutting and shuffling," *SIAM Rev.*, vol. 3, p. 293, Oct. 1961.
- [19] M. C. Pease, "An adaption of the fast Fourier transform for parallel processing," *J. Ass. Comput. Mach.*, vol. 15, pp. 252-264, Apr. 1968.
- [20] T. Lang, "Interconnection between processors and memories using the shuffle-exchange network," Stanford Univ., Stanford, Calif., Rep. R-74-19.
- [21] D. H. Lawrie, "More patterns for square blocks," Dep. Comput. Sci., Univ. Illinois, Urbana-Champaign, unpublished memo 65, July 1973.
- [22] —, "Omega networks," Dep. Comput. Sci., Univ. Illinois, Urbana-Champaign, unpublished memo 64, Apr. 1973.
- [23] U. Schumann, "Ein Verfahren zum Transponieren grosser, sequentiell gespeicherter Matrizen," *Angew. Inform.*, pp. 213-216, May 1972.
- [24] J. O. Eklundh, "A fast computer method for matrix transposing," *IEEE Trans. Comput. (Corresp.)*, vol. C-21, pp. 801-803, July 1972.
- [25] D. E. Gold and D. J. Kuck, "A model for masking rotational latency by dynamic disk allocation," *Commun. Ass. Comput. Mach.*, vol. 17, pp. 278-288, May 1974.



Duncan H. Lawrie (S'66-M'73) received the B.A. degree from DePauw University, Greencastle, Ind., the B.S.E.E. degree from Purdue University, Lafayette, Ind., and the M.S. and Ph.D. degrees in computer science from the University of Illinois, Urbana-Champaign.

He is currently an Assistant Professor of Computer Science, University of Illinois. Prior to joining the faculty, he held the position of Senior Research Programmer with the Illiac IV project at the University of Illinois, where he was in charge of language development and served as manager of the computer center. His current areas of research include the design of hardware and software for high-speed computer systems, specialized architectures for information retrieval systems, and network theory.

Dr. Lawrie serves as a referee for the IEEE TRANSACTIONS ON COMPUTERS as well as several national conferences, and as a reviewer for *Computer Reviews*. He also serves as a consultant to industry and government. He is a member of the Association for Computing Machinery, the American Association for University Professors, and several honorary societies.

System Fault Diagnosis: Masking, Exposure, and Diagnosability Without Repair

JEFFREY D. RUSSELL, MEMBER, IEEE, AND CHARLES R. KIME, MEMBER, IEEE

Abstract—Diagnosability without fault repair of a digital system containing at most t faults is considered. A system-level diagnostic model defined in an earlier paper [1] is employed. The model is to an extent independent of the means used to implement diagnostic procedures, i.e., whether the tests are accomplished via hardware, software, or combinations thereof. Two parameters, the masking and exposure indices, are defined. Conjoined with the previously defined closure index, the parameters fundamentally characterize the capability for executing valid tests in a multiple-fault environment. Necessary and sufficient conditions for a system to be t -fault diagnosable without repair are derived in terms of these parameters. Examples are presented to illustrate the application of the model for systems close to those encountered in actual practice.

Index Terms—Closure index, diagnosable digital systems, diagnostic modeling, exposure index, fault diagnosis, fault-tolerant computing, masking index, multiple faults.

I. INTRODUCTION

IN A PREVIOUS paper [1], several results concerning system-level fault detectability and diagnosability with repair were presented. Diagnosability without repair is considered in this paper. The model employed is defined in [1], and it is assumed that the reader is familiar with the model, assumptions, definitions, notations, and results given there.

The concept of t -fault diagnosability without repair is defined in the next section. Then, in Section III, the masking and exposure indices are defined. The relationships between these indices and the closure index of [1] are also established. In Section IV the main results are given which provide, in terms of these indices, necessary and sufficient conditions for a system to be t -fault diagnosable without repair.

Manuscript received April 4, 1974; revised April 29, 1975. This work was supported in part by the National Science Foundation under Grants GK-3844 and GK-35037.

J. D. Russell was with the Department of Electrical and Computer Engineering, University of Wisconsin-Madison, Madison, Wisc. 53706. He is now with Collins Avionics Division, Rockwell International, Cedar Rapids, Iowa 52406.

C. R. Kime is with the Department of Electrical and Computer Engineering, University of Wisconsin-Madison, Madison, Wisc. 53706. A portion of this research was performed while he was with the Department of Electrical Engineering and Computer Sciences, University of California, Berkeley, Calif.